

Selective betting with shrinkage — what we found

Sep 23, 2026 · @Someone

The bookmakers' closing price beat our model in every season we tested, so the best amount of shrinkage turned out to be all of it: the fitted weight came out zero in all five test seasons. Nothing we built makes money, but the bias we set out to measure is real and we measured it.

What we tested

A betting model picks the matches where it disagrees most with the bookmaker. Those are exactly the matches where the model is most likely to be wrong, because a big disagreement is usually a big error rather than a big insight. Picking on that basis therefore selects your own mistakes. Statisticians call it the winner's curse.

The proposed fix: before choosing anything, pull the model's estimate partway toward the market price. How far to pull is a weight between 0 and 1, fitted from data rather than chosen. A weight of 0 means "ignore the model, use the market"; 1 means "ignore the market, trust the model".

We tested this on 53,255 matches across 22 divisions and seven seasons, from July 2019 to May 2026. The model sees form, Elo ratings, last season's league table, an expected-goals stand-in built from shot counts, and which league the match is in. It never sees a price. Five seasons are used for testing, each predicted by a model trained only on the seasons before it.

The numbers

Log loss scores the probabilities: lower is better, and knowing nothing scores about 1.077. Accuracy is how often the most likely outcome happened. ROI is profit per unit staked, betting the model's top 10% of matches at the best price available.

	our model (trees)	our model (neural net)	the market
Five seasons, 2021/22–2025/26			
log loss	1.0145	1.0175	0.9950
accuracy	49.35%	48.97%	51.04%

	our model (trees)	our model (neural net)	the market
calibration error	0.0100	0.0161	0.0077
ROI	-6.04%	-5.03%	-2.04%
2025/26 only			
log loss	1.0186	1.0204	0.9998
accuracy	48.88%	48.86%	50.76%
ROI	-16.87%	-13.22%	-5.69%

The market column is every match bet at the best available price, with no selection at all. The model columns are their best 10% of matches. So the table says that betting everything blindly beat betting our best picks, by 4 points over five seasons and 11 points in 2025/26.

We built the whole pipeline twice, once with gradient-boosted trees and once with a neural network, to check the result was not an artefact of the learner. It was not: the two agree to within noise on every measure.

How the code works

The code runs in three stages: go and get the data, turn it into features, then train and bet. Each stage writes files the next one reads, so any stage can be re-run on its own.

flowchart TD

```

A[fetch/<br/>download raw data] --> B[build/<br/>turn it into features]
B --> C[pipeline/<br/>train, shrink, select]
C --> D[results:<br/>log loss, ROI, curves]

```

Stage	What it does	When you run it
fetch/	Downloads odds, expected goals, squad values, weather, coordinates	Once, or when a new season ends
build/	Turns raw files into one row per match with 66 features	After any fetch
pipeline/	Trains the model, shrinks it toward the market, picks bets, scores everything	Every experiment

Inside the pipeline, one match travels like this. Nothing downstream can see a price until the shrinkage step, which is the point: the model has to form its own opinion first.

```
flowchart TD
```

```
  A[one match, 66 features<br/>no prices] --> B[predict goals<br/>for each side]
```

```
  B --> C[scoreline grid<br/>goals to home/draw/away]
```

```
  C --> D[shrink toward<br/>the market price]
```

```
  D --> E[select: top 10%<br/>by disagreement]
```

```
  E --> F[settle at the<br/>best closing price]
```

The season loop is what keeps it honest. To score 2025/26, the model trains only on 2019/20 to 2024/25 and is tuned on 2024/25 alone. It never sees a match from the season it is being judged on. Then the window slides forward and the whole thing repeats — five times, once per test season.

To reproduce a run:

```
python pipeline/two_stage.py --mode multiseason --source matches_all.csv
```

Add `--export preds.csv` to save one row per match, so any season can be examined afterwards without running the whole thing again. `pipeline/two_stage_torch.py` is the same pipeline with a neural network in place of the trees, kept as a check that the results do not depend on the learner.

What the two stages learn

Stage 1 — the match engine

It does not predict who wins. It predicts **how the match is played**: expected goals for the home side, expected goals for the away side, and shots on target. Three numbers, three separate models, each trained with a Poisson loss because goals are counts rather than continuous quantities.

It sees 66 features and not one price: Elo ratings, rolling form, last season's league table, the expected-goals stand-in, and which league the match is in.

The outcome probabilities fall out afterwards, arithmetically. If the model says the home side is worth 1.6 goals and the away side 1.1, you can compute the chance of every scoreline — 0-0, 1-0, 2-1 and so on up to 10-10 — from the Poisson formula. Add up the cells where home scored more, and that is $P(\text{home win})$. The draw cells give $P(\text{draw})$, the rest $P(\text{away})$.

One correction on top: real football produces more 0-0 and 1-1 draws than two independent Poissons predict, because teams close a game down. A parameter called ρ nudges those four low-score cells, fitted per league on held-out data.

This is why stage 1 is a *match engine* rather than a classifier. The same scoreline grid gives over/under 2.5 goals and both-teams-to-score for free.

Stage 2 — the market-aware model

Stage 2 sees everything stage 1 produced **plus the price**, plus how the price moved and how much the bookmakers disagree.

It is not what we bet with. It is the yardstick: it shows how well you can do when you are allowed to look at the price, which brackets what is achievable. It scores 0.9965 against the market's 0.9950 — close, because it is largely copying the price back to us.

And that is exactly why it is not the thing we shrink. Shrinking a model that already contains the market *toward* the market is meaningless; the fitted weight would run to 1 and the correction would switch itself off while still appearing to run. So stage 1 is what gets corrected, and stage 2 stands beside it as a comparison.

The correction itself

First the closing price is turned into probabilities and the bookmaker's margin stripped out, using a method that accounts for margin being loaded more heavily onto long shots.

Then the two opinions are blended:

$$p \propto p_{\text{market}}^{1-w} \cdot p_{\text{model}}^w$$

At $w = 0$ you get the market exactly; at $w = 1$ the model exactly; in between, a weighted geometric average. The blend happens in log-odds space, which pulls hardest on the extreme probabilities — which is where the model's error is largest, and the reason this form was chosen.

w is fitted on out-of-fold predictions across every earlier season, by whichever value predicts held-out results best. We also fit it two other ways as a check: only on the matches we would actually bet, and with a different blending formula that shrinks the gap evenly instead of in log-odds. All three returned zero.

What we do with the probabilities

For each match we compute the **edge**: our probability minus the market's. Rank every candidate bet by it, take the top 10%, and place a flat stake on each at the best closing price any bookmaker offered. The cut-off is set on past seasons, never on the season being bet.

Then four things get measured: log loss and calibration on the probabilities, ROI with a bootstrap confidence interval on the bets, a risk-coverage curve showing what happens as we get pickier, and a comparison against random picks drawn at the same odds —

which is what separates "our model picks well" from "our model picks long shots".

The pipeline in notation

Every quantity has a symbol, a producer and a consumer. Follow the arrows and you can see exactly what each step receives and what it hands on.

```

flowchart TD
    X["x : 66 features<br/>form, Elo, last season, xG proxy<br/>no prices"] --> S1["stage 1 : f(x)<br/>Poisson loss"]
    S1 --> L["lamH, lamA<br/>expected goals"]
    L --> G["scoreline grid<br/>+ rho correction"]
    G --> PM["p : model probabilities<br/>(pH, pD, pA)"]
    O["o : closing average odds"] --> DV["strip the margin"]
    DV --> PI["pi : market probabilities"]
    PM --> BL["blend, weight w"]
    PI --> BL
    BL --> PC["p-hat : corrected"]
    PC --> E["e = p-hat minus pi<br/>edge per outcome"]
    PI --> E
    E --> SEL["select e >= tau<br/>top 10 percent"]
    SEL --> ST["stake 1 at best odds b<br/>profit = b-1 if win, else -1"]
  
```

The symbols

Symbol	What it is	Produced by	Fitted on
x	66 market-blind features for one match	build/ scripts	—
λ_H, λ_A	expected goals, each side	stage 1, Poisson regression	earlier seasons
ρ	low-score draw correction, one per league	fitted on held-out predictions	earlier seasons
p	model probabilities (p_H , p_D , p_A)	scoreline grid over λ and ρ	— (arithmetic)
o	closing average odds, three numbers	the odds file	—
π	market probabilities, margin removed	de-vigging o	— (arithmetic)

Symbol	What it is	Produced by	Fitted on
w	shrinkage weight, 0 to 1	fitted by log loss	earlier seasons, out-of-fold
$\hat{p}$	corrected probabilities	blend of π and p using w	— (arithmetic)
e	edge, $\hat{p}$ minus π , per outcome	subtraction	—
τ	selection threshold	90th percentile of e	earlier seasons
b	best closing odds across bookmakers	the odds file	—

Three of these are *learned* — λ , p , w . The rest are arithmetic or look-ups. That is worth noticing: the method has very few free parameters, which is deliberate.

One match, traced

Home side favoured, nothing unusual about the fixture.

1. **Features.** $x = 66$ numbers. Elo 1712 against 1655, home side averaging 1.7 goals over its last five, and so on. No price anywhere in x .
2. **Stage 1 predicts.** $\lambda_H = 1.62$, $\lambda_A = 1.14$ expected goals.
3. **Scoreline grid.** Poisson on each side gives the chance of every score from 0-0 to 10-10; p lifts the 0-0 and 1-1 cells. Summing the three regions: **$p = (0.49, 0.25, 0.26)$** .
4. **The market.** Closing average odds $o = (2.10, 3.50, 3.80)$. Their reciprocals sum to 1.025 — that 2.5% is the bookmaker's margin. Strip it: **$\pi = (0.464, 0.279, 0.257)$** .
5. **Blend.** With $w = 0.3$, each outcome gets π to the power 0.7 times p to the power 0.3, renormalised: **$\hat{p} = (0.472, 0.270, 0.258)$** . The model pulled the home price up by less than a point.
6. **Edge.** $e = \hat{p} - \pi = (+0.008, -0.009, +0.001)$.
7. **Select.** $\tau = 0.021$, read off earlier seasons. The best edge here is 0.008, well under it — **no bet**. This is the normal outcome: we act on about one match in seven.
8. **If it had cleared.** Stake 1 unit at the best closing price $b = 2.18$. Home win returns +1.18, anything else -1.

In our actual runs $w = 0$, so step 5 collapses to $\hat{p} = \pi$ and every edge in step 6 is exactly zero. That is the finding, stated in the notation: the correction is a valid mechanism that this data gives nothing to correct.

What the numbers say

The shrinkage weight is zero, in every fold. Fitted three different ways — across all matches, only on the matches we would bet, and with a different blending formula — it came out zero each time. We checked by hand that this is a real optimum and not a broken optimiser: every step away from zero makes the predictions worse. Zero means the closing price already contains everything our model knows.

The bias we set out to measure is real, and shrinkage removes it. With no shrinkage, the model expects a 127% return on the bets it picks and earns 2%. With full shrinkage it expects 10.8% and earns 6.2%. That gap between expected and realised — 125 points, falling to 5 — is the winner's curse, measured on 4,533 selected bets. The correction works exactly as the theory says.

Selection is destroying value, not adding it. Betting the top 10% by disagreement returns -6% over five seasons; betting every match at the best price returns -2%. The matches where the model most disagrees with the market are worse bets than the ones it passes over.

The method finding

This one is worth a paper on its own, and it nearly caught us out.

There are two ways to rank candidate bets. **Expected value** is probability $\times$ odds - 1, so the odds sit inside the score. **Edge** is our probability minus the market's, with no odds in it.

Ranked by expected value, our model looks like a textbook winner's curse: as it gets pickier its return falls from -5% to -9.9%. But its average odds climb from 4.39 to 9.25 at the same time. It is not selecting harder — it is drifting toward long shots, where the bookmaker's margin is fattest.

Ranked by edge, holding the prices still, the same model on the same matches is flat: -3.7% at 10% coverage, -3.5% at 5%. The decay almost disappears.

So most of what looked like selection bias was the favourite-longshot bias wearing a model's clothes. Anyone reporting selective-betting results without this check is probably reporting an odds effect and calling it selection.

What we are not claiming

Not that the market is unbeatable. We showed that *this* model does not beat it, with these features, at closing prices. That is a statement about the model.

Not that shrinkage does not work. It does what it promises: it removes the selection bias. The weight is zero only because the model has nothing to add over the price.

Not that any of this is profitable. Every arm loses money. The closest thing to a positive number came from taking the best price rather than the average, which is price comparison, not modelling — and it would not survive contact with real bookmakers, who limit or close accounts that only ever take the best price.

Still open

- **Weather** is collected for 151 of 479 grounds; the rest is one run away.
- **Measured expected goals, squad values and squad availability** exist for 23–45% of matches. They are joined but deliberately kept out of the main model, so a feature covering a quarter of the data cannot quietly shrink the study to a quarter of the data.
- **2025/26 is the worst of the five seasons** for both models, and the market's own scores got better that year. Whether the market is tightening or our model is ageing, we cannot yet separate.
- **The honest headline** is not "our model loses money". It is that selecting on a noisy score is biased, we measured the bias, and shrinking before selecting removes it — which holds whether or not anything turns a profit.